

Connector Calculus

Ghost-Node Rewrites, Bonded Liquidity, and Adversarial Agent Routing on the Lightning Network

Authors: Rubio, Dugan, and Pizarro **Program:** Spiral **Version:** Reconstructed empirical draft, August 2026

Abstract

Lightning routing is usually described as path finding on a public channel graph, but a payment succeeds on a different object: a private, directed, time-varying liquidity state. This paper develops a connector calculus for reasoning about the gap. The calculus begins with a graph-rewrite idea in which a temporary ghost node represents missing capacity or a target-value correction. A ghost edge is not itself money. It is a planning variable that must compile into an enforceable connector: a funded channel, a leased channel, or a circular rebalance. We define the channel-state algebra, conservation laws, connector types, compilation predicates, and a multi-objective potential function covering delivery, imbalance, cost, capital, and risk.

The second contribution is an economic binding layer for automated agents. A connector service is keyed to scarce economic exposure rather than to an unlimited public-key identity. It posts a bond against measurable availability and attributable failure, receives a capped reward derived from delivered service, and cannot enlarge its coalition reward by splitting into Sybil identities. This structure turns the speculative question of autonomous agent activity into a testable engineering question: can automated demand, capital, and service commitments be coupled so that agents improve rather than merely consume Lightning liquidity?

We implement a deterministic synthetic proof suite on two-community channel graphs with hidden directional balances. Across 24 paired seeds and 360 demands per seed, adding a demand-cut ghost connector raises mean payment success from 66.89% to 70.73%, a paired increase of 3.84 percentage points with a normal 95% interval of approximately ± 0.49 points. A random cross-cut connector reaches 70.53%; the ghost rule exceeds it by only 0.20 points, with an interval that includes zero. The strong result is therefore connector value, not optimal endpoint selection. Under a bounded synthetic lock attack on high-betweenness edges, the ghost connector raises success from 50.49% to 67.60%, a paired gain of 17.12 ± 1.66 points. A separate reward experiment confirms exact invariance when one capital source is split across 1 to 32 identities. These experiments establish mechanism evidence, not mainnet performance claims. No live node, wallet, transaction broadcast, or local zero-knowledge prover is used.

Keywords: Lightning Network, payment channels, graph rewrite, ghost node, liquidity, routing, bonding, Sybil resistance, autonomous agents, network flow

1. Introduction

The Lightning Network turns a set of bilateral payment channels into a multi-hop payment network [Poon and Dryja 2016; BOLT specifications 2026]. Its public routing object contains nodes, advertised channel capacities, fee schedules, timelocks, and feature information. Its operational object additionally contains the directional balance of each channel, pending HTLCs, local reserves, peer behavior, and knowledge accumulated from previous attempts. Those additional variables determine whether an advertised route can carry a payment at a particular moment. They are not globally public, and revealing them would create privacy and strategic costs.

This mismatch has two consequences. First, a topologically connected network can be financially disconnected for an amount and direction. Second, routing is inseparable from liquidity management. A route attempt updates local beliefs even when no payment is delivered; a successful payment changes the balances that make later routes feasible. Path selection, probing, rebalancing, channel opening, leasing, fees, and privacy are therefore different operations on one evolving state.

Existing work gives strong pieces of this picture. Pickhardt and Richter formulate reliable multi-part payments as probabilistic minimum-cost flows [Pickhardt and Richter 2021]. Pickhardt and coauthors quantify the privacy-efficiency tradeoff created by uncertain balances [Pickhardt et al. 2021]. Other studies show practical balance probing, timing leakage, cross-layer deanonymization, centrality concentration, jamming, and grieving risks [Tikhomirov et al. 2020a; Nisslmueller et al. 2020; Romiti

et al. 2020; Zabka et al. 2022; Shikhelman and Tikhomirov 2022; Mazumdar et al. 2022]. The BOLT documents themselves acknowledge that routing must consider fee, timelock, diversification, privacy, and local balance [BOLT specifications 2026].

This paper adds a compact algebra for the intervention between diagnosis and execution. The starting intuition comes from the earlier work titled *Graph Re-write Using Ghost Nodes for Target Value Rebalancing* [Rubio, Dugan, and Pizarro 2020]. A ghost node makes a missing balancing action explicit inside an augmented graph. In the present setting, the ghost can supply a candidate edge across a directional cut, absorb an excess, or close a cycle. The augmented solution is useful only if each nonphysical edge compiles into a real action with sufficient capital, timelock, policy, and accountability.

The term **connector** denotes that compiled action. A connector can change topology, change directional state without changing topology, lease capacity for a horizon, or state a cross-system settlement contingency. This vocabulary separates four operations that are often blurred together. Opening a new channel increases off-chain capacity. A circular rebalance moves existing balances. A lease temporarily commits capacity and introduces counterparty and expiry terms. A virtual connector depends on a settlement adapter such as an Ark VTXX or an asset protocol; it is not executable merely because an optimizer draws an edge.

The paper also asks what changes when software agents become persistent economic actors rather than passive wallet features. Agents can issue repeated payments, adapt their routing from failures, coordinate across services, and allocate capital faster than humans. The same capabilities can produce useful liquidity discovery or abusive probing, wash flow, jamming, identity splitting, and correlated topology capture. We avoid anthropomorphic claims and do not assume a literal hive mind. We model a swarm as a set of policy processes that may share objectives, information, capital, or control.

The practical thesis is that agent demand can help solve liquidity only when demand and service are bonded. Unpriced activity merely consumes channel state. A useful agent connector commits scarce capital, accepts a bounded service obligation, and earns from measurable delivery rather than from identity count or message volume. This does not make every failure slashable; routing is uncertain and failures can be outside an operator's control. It does create a formal place to express availability, attributable fault, reward caps, and evidence.

1.1 Contributions

The work makes six contributions.

1. It defines a directional channel-state algebra with explicit conservation, reserve, lock, fee, and route-feasibility terms.
2. It defines topological, circular, leased, and virtual connector types and a ghost-to-connector compilation rule.
3. It proposes a potential function that keeps delivery improvement distinct from imbalance, cost, capital, and risk.
4. It defines a capital-keyed bonding and reward rule whose aggregate payout is invariant to identity splitting.
5. It specifies a defensive adversary model covering Sybil splits, jamming, wash flow, probing, false availability, collusion, and concentration.
6. It supplies a reproducible synthetic proof suite, machine-readable proof cards, and a bounded evidentiary interpretation.

1.2 Claims and non-claims

The mathematical claims concern the model: conservation, feasibility, compilation, and identity-split invariance. The empirical claims concern the checked simulator configuration. We do not claim that a 3.84-point synthetic gain predicts a mainnet gain. We do not claim that the demand-cut heuristic is an optimal channel-opening rule. We do not implement production HTLC timing, multi-part atomicity, on-chain fees, peer scoring, Ark settlement, TradeLayer assets, or zero-knowledge proofs. The experiments do not contact the Lightning Network and cannot be used to conduct a real attack.

2. Background and Related Work

2.1 Public channels and private liquidity

A Lightning channel locks a total capacity on Bitcoin and maintains a sequence of enforceable off-chain states. Public gossip can advertise the channel and its directional relay policies, including base fee, proportional fee, minimum and maximum HTLC values, and CLTV delta. The balance available in a direction is not part of the public channel announcement. BOLT 7 therefore describes a routing graph whose edges may be unusable for the requested amount even when the channel is active [BOLT specifications 2026].

Suppose a channel between nodes u and v has capacity c . Let b_{uv} be the amount that u can currently push toward v in the abstraction, and let $b_{vu} = c - b_{uv}$. A payment from u toward v decreases b_{uv} and increases b_{vu} . Capacity is conserved, but

directionality changes. A sequence of one-way payments can strand capacity on the wrong side of a cut while every channel remains open.

The hidden balance is not a mere missing database field. Publishing it would aid routing while also enabling surveillance and strategic targeting. Probing research demonstrates that error behavior and trial payments can reveal balance information [Tikhomirov et al. 2020a]. Probabilistic routing work treats uncertainty directly and shows that reliability, expected attempts, and privacy are coupled [Pickhardt et al. 2021]. The connector calculus preserves that distinction by keeping public capacity C and private state B as separate components.

2.2 Flow, multi-part payments, and uncertainty

Classical maximum-flow theory asks how much commodity can move through a capacitated network [Ford and Fulkerson 1956]. Lightning adds fees, uncertain directional capacities, timelocks, discrete HTLC constraints, path privacy, and state changes after each payment. Pickhardt and Richter show that probabilistic reliability costs can be incorporated into separable convex-cost flow computations and that minimum-cost flow generalizes path search to multi-part payments [Pickhardt and Richter 2021].

Our model is complementary. It does not replace a routing algorithm. It gives a typed language for changing the state on which routing operates. A solver may diagnose a directional cut deficit. The connector layer asks whether to add capital, lease capital, or move existing balances around a cycle; what that action costs; which constraints it must satisfy; and which party bears failure risk.

2.3 Privacy, centrality, and attack surface

Lightning's onion format reduces what each hop learns, but topology, timing, failure, channel, and cross-layer information remain observable in different threat models. Research has described active balance probing and passive timing attacks [Nisslmueller et al. 2020], links between Lightning nodes and on-chain Bitcoin activity [Romiti et al. 2020], and concentration in transaction-relevant centrality [Zabka et al. 2022]. Sharma and coauthors show that a strategically placed colluding minority can sharply reduce sender anonymity in modeled networks [Sharma et al. 2022].

Jamming exploits scarce in-flight HTLC or liquidity resources. The attacker can create payments that occupy resources and fail or delay completion, harming unrelated users at a lower economic cost than the blocked value. Shikhelman and Tikhomirov argue for evaluating countermeasures across security, privacy, user experience, and incentive compatibility, and favor local, behavior-based reputation in light of Sybil risk [Shikhelman and Tikhomirov 2022]. Griefing analyses likewise emphasize the difficulty of pricing delay and assigning penalties with current scripting constraints [Mazumdar et al. 2022].

These findings motivate three design choices. First, a connector score cannot be only delivered volume. Second, reputation cannot be a global identity vote. Third, a bond may cover only explicitly attributable, objectively witnessed service failures; it cannot magically resolve ambiguous network causality.

2.4 Ghost-node graph rewrites

The earlier ghost-node work proposes graph rewrites for target-value rebalancing [Rubio, Dugan, and Pizarro 2020]. The recovered project did not retain the application text, so this paper does not reproduce or extend its claims. It adopts only the high-level construction: augment a graph with a nonphysical node or edge, solve a balancing problem in that augmented space, then map the result back to permissible operations.

This separation is valuable because an optimizer can otherwise hide assumptions. Drawing an edge with 100,000 satoshis of capacity silently assumes an owner, funding output, counterparty, fee, availability horizon, and failure rule. Calling the edge a ghost makes its provisional status visible. Compilation must supply the missing economic and protocol data.

3. System Model

3.1 Channel state

At discrete step t , define the bounded network state as:

Equation

$$S_t = (V, E_t, C, B_t, P_t, L_t, K_t).$$

V is the node set. E_t is the set of channels. C assigns a positive total capacity c_e to each channel. B_t assigns directional balances $b_{uv}(t)$ and $b_{vu}(t)$ to the two orientations of $e = \{u, v\}$. P_t contains relay policy variables such as base fee, proportional fee, and CLTV delta. L_t contains temporarily locked balance. K_t contains active connectors and their capital, bond, and expiry metadata.

For every channel:

Equation

$$\begin{aligned} b_{uv}(t) + b_{vu}(t) &= c_e, \\ 0 &\leq b_{uv}(t) \leq c_e, \\ 0 &\leq l_{uv}(t) \leq b_{uv}(t). \end{aligned}$$

Let r_e be a local reserve abstraction. The amount available from u to v is:

Equation

$$a_{uv}(t) = \max(0, b_{uv}(t) - r_e - l_{uv}(t)).$$

The implementation aggregates parallel channels between a pair into one edge. Production systems must retain channel identity because fee policy, lock state, and failure domain may differ.

3.2 Demands and routes

A demand is $d = (s, z, q, h)$, where s is the sender, z the receiver, q the amount, and h an optional deadline or service horizon. A path $p = (v_0, \dots, v_k)$ is feasible for q if every oriented edge has available balance at least q and the accumulated policy constraints fit the request.

Equation

$$\text{Feasible}(p, q, S_t) \text{ iff } q \leq a_{(v_i, v_{i+1})}(t) \text{ for all } i, \text{ and Policy}(p, q) \text{ is satisfied.}$$

The bounded simulator uses one-part payments and additive fees. It does not model backward amount accumulation, HTLC slots, or atomic multi-part settlement. These simplifications make the conservation mechanics transparent but limit protocol interpretation.

3.3 Payment rewrite

If a feasible amount q traverses oriented channel u to v , the channel rewrite is:

Equation

$$T_{(u, v, q)}: (b_{uv}, b_{vu}) \rightarrow (b_{uv} - q, b_{vu} + q).$$

For a path, compose the edge rewrites in path order. The route is validated before any edge is changed, making the operation atomic in the simulator. Relay fees are recorded as cost but not transferred into intermediate channel balances; a production model would include the slightly different amount at each hop.

3.4 Imbalance energy

We need a scalar that reports directional skew without pretending that 50/50 is always economically optimal. Define a diagnostic energy:

Equation

$$I(S_t) = (1 / \sum_e c_e) * \sum_e c_e * ((b_{uv} - c_e/2) / (c_e/2))^2.$$

I is zero only when every channel is balanced. It rises toward one as channels become one-sided. This is a health indicator, not the routing objective. A profitable node with asymmetric customer flow may rationally maintain asymmetric balances. Demand-weighted target balances can replace $c_e/2$ without changing the algebra.

3.5 Observability levels

The model distinguishes three routing knowledge modes.

Mode	Balance knowledge	Attempts	Interpretation
Public	Public graph and policy only	One cheapest candidate	Low active leakage, lower adaptation
Adaptive	Public graph plus prior failures	Up to eight candidates	Better delivery, more failure observations
Oracle	Exact simulator balance state	One feasible candidate	Unattainable upper information condition

The oracle is not a proposed API. It provides a reference point. The number of failed candidates is recorded as a leakage proxy because each differentiated failure may reveal something about the route. It is not an entropy estimate or deanonymization measurement.

4. Connector Calculus

4.1 Connector types

A connector is a typed state transition $\kappa = (\text{type}, \text{endpoints}, q, \text{bond}, \text{expiry}, \text{capital_id})$.

Type	State effect	Capital effect	Typical realization
Topological	Adds or expands an edge	Adds committed channel capital	Funded channel or splice
Circular	Changes directional balances along a closed route	Conserves total channel capacity	Circular rebalance
Leased	Adds an edge for a horizon	Commits temporary capacity and bond	Liquidity lease or dual-funded service
Virtual	Declares contingent cross-system capacity	Depends on external settlement	Ark VT XO, swap, or asset adapter

The types prevent category errors. A circular rebalance cannot repair a disconnected topology. A topological connector cannot be treated as costless. A lease cannot be assumed after expiry. A virtual connector cannot execute until an adapter proves its settlement and exit conditions.

4.2 Ghost plans

A ghost plan is:

Equation

$$g = (\text{id}, \text{source}, \text{target}, q, \text{mechanism}, H, \text{rho}),$$

where H is the service horizon and rho is a risk rate. It is inserted into an augmented optimization graph as if q units could move from source to target. The plan is a counterfactual: it tells us how valuable that capacity would be under the modeled demand.

Compilation creates a real connector only if:

1. the mechanism is supported;
2. q is backed by unique economic exposure;
3. both channel endpoints or service parties consent;
4. the funding and fee budget is available;
5. the expiry and exit rules are explicit;
6. the required bond is posted; and
7. the connector does not violate reserve, concentration, or policy caps.

The reference implementation uses a simple required bond:

Equation

$$B_g = \text{ceil}(q * (\alpha + \text{rho} * H)),$$

with α a capital fraction. This is an illustrative rule. A deployment would use a bounded loss model, on-chain fee risk, service value, and fault attribution.

4.3 Cut completion

Let A be a subset of nodes and $A\text{-bar}$ its complement. Define the currently available directed capacity across the cut:

Equation

$$Q_A(t) = \sum_{(u \in A, v \in A\text{-bar})} a_{uv}(t).$$

Let forecast demand across the cut over horizon H be $D_A(H)$. A directional deficit exists when $D_A(H)$ exceeds Q_A after the operator's reserve and confidence margin. A ghost solver may add candidate edges from high-demand sources in A to high-demand destinations in $A\text{-bar}$, estimate the reduction in delivery loss, and retain candidates whose value exceeds their capital, fee, and risk cost.

Our demand-cut heuristic is intentionally primitive: it observes the first quarter of the synthetic demand stream, chooses the most frequent cross-cluster source-destination pair, and proposes a 120,000-satoshi channel. It does not optimize betweenness, expected fees, on-chain cost, counterparty reliability, or future demand. The random control adds the same nominal capital between random nodes on opposite sides of the cut.

4.4 Potential function

A connector policy should not maximize success in isolation. Define:

Equation

$$\begin{aligned} \text{Phi}(S, K; D) = & \lambda_d * \text{DeliveryLoss}(S, K; D) \\ & + \lambda_i * I(S) \\ & + \lambda_f * \text{ExpectedFees}(S, K; D) \\ & + \lambda_c * \text{LockedCapital}(K) \\ & + \lambda_r * \text{RiskExposure}(K). \end{aligned}$$

Connector selection seeks a feasible K that reduces Phi . The weights are policy choices, not universal constants. A merchant may put high weight on inbound reliability. A routing node may value fee revenue and balanced reuse. A privacy-sensitive wallet may penalize adaptive failures. A risk controller may cap exposure to a correlated operator even when expected delivery is high.

The key methodological point is vector reporting. In the experiments, the ghost connector improves success while ending with higher imbalance energy than baseline. It would be misleading to call that an unconditional improvement. Extra delivery uses the new and old channels asymmetrically. A later rebalance or a different target balance may be justified, but the success metric alone does not prove it.

5. Algebraic Properties

Proposition 1: Channel conservation

For any feasible payment rewrite $T_{(u,v,q)}$, channel capacity is invariant.

Equation

$$(b_{uv} - q) + (b_{vu} + q) = b_{uv} + b_{vu} = c_e.$$

By composition, a feasible path payment conserves every traversed channel's capacity and therefore total off-chain capacity. It changes ownership-side balance, not channel capacity.

Proposition 2: State bounds

If $q \leq a_{uv} \leq b_{uv}$, then $b_{uv} - q \geq 0$. Since $q \leq b_{uv}$ and $b_{vu} = c_e - b_{uv}$, the new reverse balance $b_{vu} + q \leq c_e$. A prevalidated path therefore remains inside the channel-state polytope.

Proposition 3: Closed-route node conservation

For a circular connector whose route begins and ends at the same controlled node, every internal route node receives and forwards the same modeled amount q . The route changes directional channel balances but introduces no new channel capacity. Fees and amount variation are omitted in the bounded proof; a production cycle uses hop-adjusted amounts and pays a net fee from the initiating wallet.

Proposition 4: Ghost non-executability

A ghost plan alone does not change S . Only $\text{Compile}(g)$ followed by a supported connector transition can change E , B , or K . This prevents an optimization artifact from being counted as funded liquidity.

Proposition 5: Identity-split invariance

Let capital source c have service weight w_c , and let the total reward pool be R . Allocate capital reward:

Equation

$$R_c = R * w_c / \sum_j w_j.$$

If m identities claim the same capital source, divide R_c among them. Their coalition sum remains R_c for all $m \geq 1$. Identity splitting changes labels and individual shares, not aggregate reward. This property fails for a naive per-identity grant.

The proofs appear in expanded form in Appendix A and are exercised in unit and randomized tests.

6. Bonding Automated Connector Operations

6.1 Why agents change the problem

Automation compresses the interval between observation and action. A persistent agent can watch local success, fees, peer behavior, inventory, and demand; decide whether to rebalance or acquire inbound liquidity; negotiate or select a service; and repeat. A population of such agents can create feedback that no single wallet controls. Some feedback is useful: fees signal scarcity, failed routes update beliefs, and service commitments place capacity where demand is demonstrated. Other feedback is harmful: correlated agents can chase the same yield, amplify a false signal, or concentrate flow on a narrow connector set.

The phrase **agent swarm** is therefore defined operationally. It means multiple policy processes with possibly shared objectives, information, software lineage, or capital control. No claim about consciousness or autonomous legal personality is required. The relevant unit for risk is the control and capital domain, not the number of keys.

6.2 Service contract

A bonded connector service declares:

- capital identifier and amount;
- endpoint or cut served;
- availability horizon;
- maximum accepted flow or number of concurrent obligations;
- pricing rule;
- objective witness events;
- bond amount and slash schedule;
- expiry and exit path;
- privacy boundary; and
- concentration group or correlated failure domain.

The capital identifier need not reveal a civil identity. It must prevent the same scarce resource from being counted repeatedly. A proof of control over a funding output, channel point, or contract position can serve this purpose, subject to privacy design. Consensus over global reputation is not required. Each customer or routing peer can maintain local service evidence.

6.3 Bond settlement

The implementation scores delivered volume and availability, then applies bounded slashing for downtime and attributable failures. In the worked configuration, both example operators post 25,000 units. The reliable operator is available for 98 of 100 steps, delivers 470,000 of 500,000 attempted units, and has one attributable failure. It releases 24,749 and loses 251, with a score of 0.946. The unreliable operator is available for 62 steps, delivers 220,000 units, and has 12 attributable failures. It releases 21,125 and loses 3,875, with a score of 0.357.

These numbers illustrate discrimination, not a recommended production schedule. Aggressive slashing can itself be attacked through false failure attribution or induced congestion. The service must define evidence that the provider could control. Ambiguous network failures should affect local selection beliefs before they affect collateral.

6.4 Reward cap and Sybil resistance

Permissionless keys are cheap. A rule that pays each connector identity a fixed bonus invites arbitrary multiplication [Douceur 2002]. Our rule groups claims by economic capital identifier, chooses one weight per capital source, allocates a fixed pool across sources, and divides a source's reward among identities claiming it.

This does not solve every Sybil problem. One operator may control genuinely distinct UTXOs and channels. Correlated capital can still dominate a market. A deployment needs group caps, diminishing returns, and diversity constraints based on topology, hosting, implementation, or counterparty domain. The narrow theorem is valuable because it removes the easiest identity-only amplification.

7. Defensive Adversary Model

The red-team model asks how automated policies can exploit the connector system and what evidence would reveal failure. It deliberately omits instructions for hiding transactions or evading monitoring.

7.1 Sybil splitting

An operator creates many public keys or service records to collect identity-based rewards, gain reputation votes, or appear diversified. The primary control is capital-keyed aggregation. Secondary controls include local reputation, group exposure caps, and diminishing marginal reward.

7.2 Liquidity mirage

A service advertises capacity that is unavailable, simultaneously promised elsewhere, or likely to disappear before the stated horizon. Compilation requires proof of unique economic exposure and explicit expiry. Customers should distinguish proof of capital from proof of usable directional balance; the former does not imply the latter.

7.3 Jamming and griefing

An adversary occupies scarce in-flight or balance resources using payments that delay or fail. The bounded experiment models this only as a 90% lock on the two highest edge-betweenness channels. It does not create HTLCs or represent the full protocol. Controls include per-peer resource accounting, unconditional attempt fees where appropriate, local behavior scores, connector diversity, and avoiding concentration on a single cut.

7.4 Probing and adaptive leakage

A router can learn from differentiated failures. Repeated adaptive attempts improve delivery but expose a larger observation surface. Controls include limiting retries, coarsening failure information, randomizing path choice, payment splitting, blinded paths where supported, and treating information acquisition as a cost in Φ . The experiments report failed attempts as a proxy, not as an attack recipe.

7.5 Wash flow and reward farming

Colluding agents send circular or reciprocal payments to manufacture volume and collect service rewards. Delivered nominal volume is therefore not sufficient evidence. Reward should depend on externally originated demand, net cut service, fee expenditure, uniqueness constraints, or customer diversity. Circular rebalances should be classified separately from user delivery.

7.6 Collusive connector capture

Several apparently distinct connectors share control and occupy the same topological cut. They can censor, surveil, or jointly withdraw. Capital-keyed Sybil resistance does not ensure path diversity. The policy must cap correlated cut exposure and reward edge-disjoint alternatives.

7.7 Steganographic coordination risk

Payment timing, amount classes, invoice behavior, success/failure patterns, and route choice can act as a coordination surface even when payloads are onion-encrypted. A population of agents could infer shared state from ordinary economic events. This observation does not imply a reliable covert channel, and this paper does not design one. It implies that safety analysis cannot inspect message payload alone. Monitors should look for correlated control loops, repeated low-economic-value patterns, synchronized liquidity movements, and reward outcomes inconsistent with independent demand.

7.8 Model poisoning and demand spoofing

An attacker generates early demand to induce a ghost solver to fund a poor connector, then redirects real flow or abandons the region. Controls include holdout windows, minimum observation horizons, adversarially robust demand estimates, capital-at-risk for signal providers, and post-deployment counterfactual evaluation.

8. Experimental Design

8.1 Research questions

The proof suite addresses five bounded questions.

- **RQ1:** Do implemented payment rewrites preserve capacity and channel bounds?
- **RQ2:** Does adding a cross-cut connector improve delivery under skewed demand?
- **RQ3:** Does the simple demand-cut endpoint rule outperform equal-capital random cross-cut placement?
- **RQ4:** Does connector diversity reduce damage in a synthetic high-centrality lock scenario?
- **RQ5:** Is coalition reward invariant when one capital source splits across identities?

8.2 Topology and state generation

Each seed generates two clusters of eight nodes. Every cluster receives a cycle and up to four random chords. Internal channel capacities are sampled from bounded integer ranges. Two thin cross-cluster channels connect A0-B0 and A4-B4 with capacities 45,000 and 55,000 satoshis and deliberately weak A-to-B balances of 9,000 and 13,000 satoshis. All arithmetic that changes balances is integer arithmetic.

Each seed produces 360 demands. Fifty-eight percent move from cluster A to B, 15% from B to A, and 27% remain inside a cluster. Amounts are sampled from 1,000, 2,000, 5,000, and 8,000 satoshis. The skew depletes the thin A-to-B cut and creates the intended balancing problem.

8.3 Routing modes

The adaptive baseline enumerates up to eight shortest simple paths ordered by public additive fee and hop count. It tries candidates until one is feasible. The public mode tries only the cheapest public candidate. The oracle filters candidates by exact simulator balance before choosing. Every successful route atomically updates directional balances.

8.4 Connector interventions

The random intervention adds a 120,000-satoshi channel between random nodes in opposite clusters. If it duplicates an existing abstracted pair, no channel is added, which is why its mean added capital is 115,000 rather than 120,000. The ghost intervention examines the first quarter of demands, selects the most frequent cross-cluster pair, and adds a 120,000-satoshi channel initialized 50/50.

8.5 Jamming abstraction

The lock experiment computes edge betweenness on the post-intervention topology, selects the two highest-scoring edges, and marks 90% of each directional balance unavailable. It then runs the same demand stream. This tests path diversity under resource removal. It is not an HTLC-level jamming simulator and yields no claim about attack cost.

8.6 Sybil and bond experiments

The Sybil experiment gives one 100,000-unit capital source a 0.9 service score and splits its claim across 1, 2, 4, 8, 16, and 32 identities. The invariant policy distributes a fixed 100,000-unit pool once per capital source. The naive control pays 100,000 units per identity.

The bond example compares two fixed service records under the same 25,000-unit posted bond. It validates monotonic discrimination: worse observed availability and more attributable failure produce greater slashing and a lower score.

8.7 Reproducibility and proof cards

The suite uses seeds 0 through 23, deterministic Python random generators, NetworkX path enumeration, and no network I/O. It writes raw connector rows, a JSON summary, three figures, and proof cards. Ten unit tests cover conservation, atomic infeasibility, ghost compilation, deterministic simulation, bonding discrimination, and identity-split invariance.

9. Results

9.1 Algebraic invariant campaign

The randomized invariant campaign attempted 600 payments on one generated state. It applied 599 feasible routes and rejected one. After every applied route, it checked each channel sum, directional bounds, lock bounds, and total capacity. All checks passed. This is implementation evidence for Propositions 1 and 2, not formal verification of the Python interpreter.

9.2 Connector completion

Strategy	Mean success	Mean volume delivery	Mean failed-attempt proxy	Mean ending imbalance	Mean added capital
Baseline	66.89%	58.09%	2.886	0.228	0
Random cross-cut	70.53%	61.73%	2.690	0.321	115,000
Demand-cut ghost	70.73%	62.05%	2.666	0.307	120,000

The ghost connector improves success over baseline by 3.8426 percentage points. The paired normal 95% half-width is 0.4923 points, so the interval is approximately [3.35, 4.33] points. The added connector also raises delivered-volume share by about 3.96 points and lowers failed attempts relative to baseline.

The endpoint-selection claim is weaker. The ghost connector exceeds the random cross-cut control by 0.1968 points, but the paired half-width is 0.6213 points. The interval includes zero. On this simple two-community topology, most of the value comes from adding cross-cut capital, not from the observed-frequency rule. A stronger placement claim requires richer graphs, on-chain cost, holdout demand, and a control matched on successful channel creation.

The ending imbalance energy is higher with either connector than without one. More demands succeed and move balance in the dominant A-to-B direction. This illustrates why delivery and balance must be reported separately.

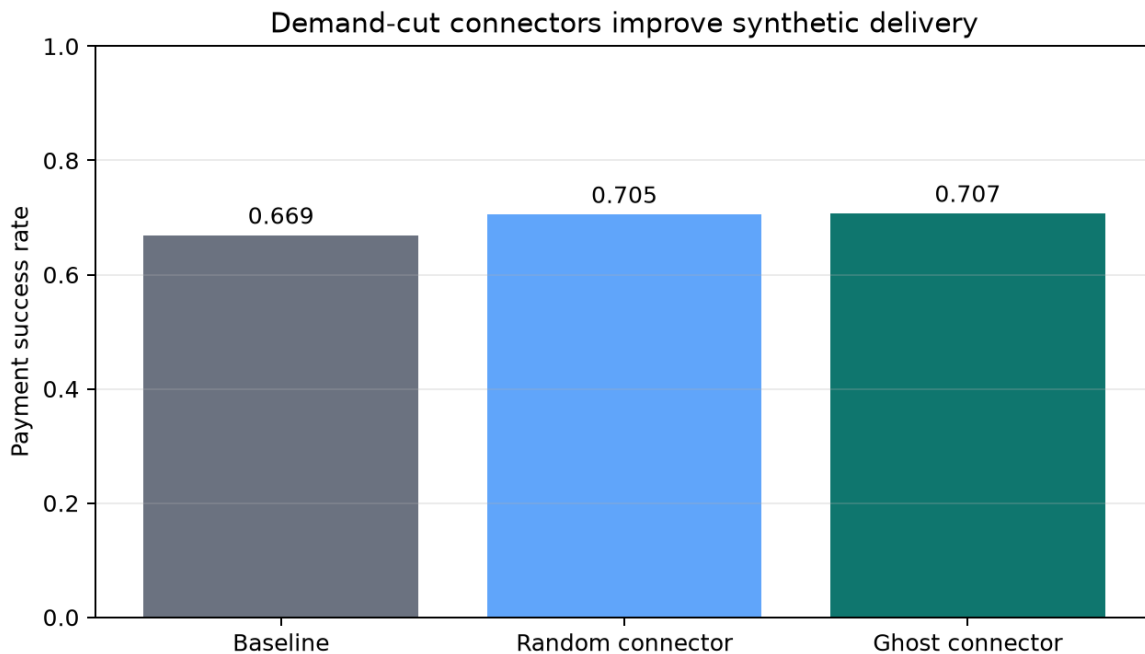


Figure. Connector success.

9.3 Privacy-efficiency reference

Knowledge mode	Mean success	Mean volume delivery	Failed-attempt proxy
Public, one candidate	64.42%	56.20%	0.356
Adaptive, up to eight	66.89%	58.09%	2.886
Oracle feasibility	66.89%	58.09%	0.331

Adaptive retries gain 2.47 success points over the one-candidate public mode but generate far more failed observations. The oracle matches adaptive delivery with fewer failed attempts because it has information that real senders do not possess. The result reproduces the direction of the privacy-efficiency tension identified in prior research, but the proxy is too simple for quantitative privacy comparison.



Figure. Privacy frontier.

9.4 Synthetic lock resilience

Without added connector capital, locking 90% of directional balances on the two highest-betweenness edges lowers mean success to 50.49% and delivered volume to 38.31%. With the demand-cut connector, mean success is 67.60% and volume delivery is 58.63%. The paired success gain is 17.1181 points with a 95% half-width of 1.6649 points.

This is the strongest empirical result because the added edge creates an alternate cut path and changes which edges are central. It is also the result most sensitive to the abstraction. Real jamming is constrained by HTLC counts, timeouts, payment sizes, fees, peer controls, and attacker capital. The experiment supports the general value of connector diversity under edge unavailability, not a numerical claim about mainnet defense.

9.5 Sybil-invariant reward

Identities	Capital-keyed coalition reward	Naive per-identity reward
1	100,000	100,000
2	100,000	200,000
4	100,000	400,000
8	100,000	800,000
16	100,000	1,600,000
32	100,000	3,200,000

The capital-keyed pool is exactly invariant across all tested identity counts. The naive control scales linearly. This is a constructive proof of a narrow but important property: identity multiplication alone cannot mint additional reward.

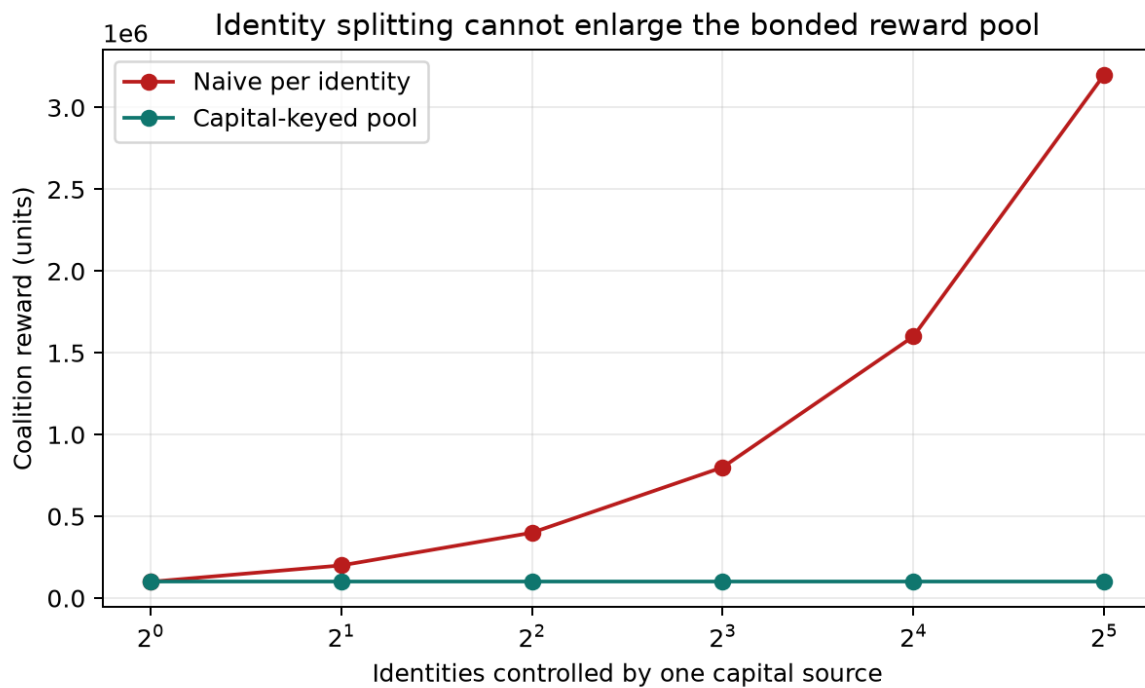


Figure. Sybil rewards.

9.6 Proof-card disposition

Card	Claim	Disposition
P1	Payment rewrites conserve channel capacity	Supported by 599 applied randomized routes and unit tests
P2	A demand-cut connector improves delivery over the unmodified graph	Supported in the specified synthetic topology
P2b	The demand-cut endpoint rule beats random cross-cut placement	Not supported at 95% in this experiment
P3	Capital-keyed reward is invariant to identity splitting	Supported analytically and for 1-32 identities
P4	Bond settlement discriminates reliable from unreliable service	Supported for the worked records; schedule remains illustrative

10. Implications for AI Agents and Lightning Liquidity

10.1 From clandestine demand to accountable service

The motivating concern was that AI agents may use Lightning as a payment and coordination substrate while conducting activity that is difficult for any single observer to interpret. Onion routing, private balances, repeated invoices, and rapid autonomous action create a plausible environment for opaque coordination. The right response is not to assume that all automated flow is malicious or to demand global identity. It is to separate payment privacy from service accountability.

An agent may keep its strategy and customer relationships private while still proving that connector capital is unique, that a service window existed, that a quoted fee rule was followed, and that a bounded failure event occurred. A connector receipt can commit to configuration and result hashes without exposing prompts, model identifiers, strategy payloads, or unrelated payments. Accountability attaches to the economic action, not to a biography of the software.

10.2 Endogenous liquidity loop

A constructive agent-liquidity loop has five stages.

1. Private demand produces local success, fee, and failure observations.
2. Aggregated observations reveal a persistent directional cut deficit without publishing individual payments.
3. A ghost solver prices candidate capital or rebalance actions against expected delivery gain.
4. An agent compiles a selected candidate by committing capital and bond under a service horizon.
5. Post-deployment receipts compare promised and observed service, update local selection, and release or slash bounded collateral.

This loop can make liquidity more endogenous: the actors generating demand also create a price signal and may supply capital. It does not eliminate the need for human capital owners, LSPs, routing nodes, or on-chain transactions. It provides a common control surface for them.

10.3 Why clandestine activity does not automatically solve liquidity

More traffic can worsen liquidity when it is one-directional, bursty, or adversarial. Hidden agent flow does not become socially useful merely because it pays fees. It may deplete a cut, reveal balances through retries, or reward a central intermediary. The connector calculus points to necessary conditions for positive contribution:

- the agent commits or pays for scarce capacity;
- rewards are tied to net service rather than gross self-generated volume;
- the service survives identity splitting and wash-flow controls;
- connector diversity is measured across failure domains;
- privacy cost and retry leakage enter the objective; and
- exit and expiry are explicit.

Thus, bonding agent operations to liquidity is not a claim that autonomous finance should be clandestine. It is a proposal to convert opaque demand into priced, capped, and auditable service obligations.

10.4 Policy separation

Three policy layers should remain distinct.

Routing policy selects paths for a payment. **Liquidity policy** decides when to rebalance, open, lease, or close capacity. **Risk policy** limits peers, correlated exposure, retries, bonds, and external adapters. A single optimizing agent may implement all three, but the interfaces and evidence should be separate. This makes it possible to replace a model without changing settlement logic and to halt a risky connector without disabling ordinary payments.

11. Cross-System Connectors: Ark, Assets, and ZK Boundaries

11.1 Ark and VTXO connectors

Ark represents off-chain value using virtual transaction outputs that can ultimately be materialized through Bitcoin transactions [Ark protocol contributors 2026]. The term **connector** also appears in Ark's technical design, but the present calculus uses the word generically. A virtual connector from a Lightning cut to an Ark VTXO would require an adapter defining ownership proof, swap or settlement path, timeout ordering, unilateral exit, fee exposure, and failure attribution.

The algebra can represent the contingent edge and its risk cost. It cannot establish atomicity by notation. A future experiment should use regtest and a specific Ark implementation, with no assumption that VTXO availability equals Lightning inbound balance.

11.2 Asset protocols and TradeLayer

An asset-denominated connector may post collateral or settle rewards in a non-BTC asset. This expands the state with exchange-rate, oracle, liquidation, and chain-reorganization risk. The safest initial integration is accounting-only: record a connector's BTC service evidence, then settle an external reward through an independently specified adapter. Embedding arbitrary agent payloads, strategy identifiers, or prompts in payment metadata is unnecessary and creates privacy and attack surface.

11.3 Zero-knowledge proofs

Zero-knowledge proofs could eventually show that a service calculation followed a committed rule without revealing every route. They are not required for the core algebra. The recovered project recorded repeated local out-of-memory failures during ZK attempts, so this reconstruction performs no proving. It emits hashable JSON proof cards and receipts. A later remote or specialized proving campaign should begin only after the non-ZK statement, witness schema, memory budget, and verification benefit are fixed.

12. Limitations and Validity

12.1 Synthetic topology

The topology is deliberately small and structured. It makes a cut deficit easy to interpret but favors any cross-cut channel. Mainnet graphs contain heterogeneous capacity, parallel channels, private edges, changing policies, offline nodes, implementation differences, and strategic operators. The endpoint heuristic must be tested on time-sliced public snapshots with simulated hidden balances before practical claims.

12.2 Simplified payments

Every hop moves the same integer amount. Real senders construct amounts backward so upstream hops include downstream fees. The simulator omits HTLC minimum and maximum values, slots, CLTV accumulation, MPP atomicity, AMP, trampoline routing, route blinding, mission control, and peer-specific channel selection. These omissions matter most for fee and jamming conclusions.

12.3 Balance initialization and demand

Initial directional balances are sampled from bounded ranges, and cross-cut imbalance is hand-constructed. Demand is independent across steps except for a fixed mixture. Real demand has daily cycles, merchants, recurring counterparties, exchange flows, correlated failures, and strategic adaptation. A connector selected from early demand can be poisoned or become obsolete.

12.4 Privacy proxy

Failed attempts are not equivalent to learned bits. Failure messages may be coarsened, ambiguous, or observed by different parties. The privacy figure is a conceptual frontier. Rigorous work should define an adversary, prior distribution, observation channel, and posterior entropy or guessing advantage.

12.5 Bond attribution

The bond example assumes objectively attributable failures. In a multi-hop network, causality is often disputed. A provider can be framed by upstream behavior, a customer can submit malformed attempts, and a chain event can affect everyone. Production slashing requires cryptographically bound quotes, timestamps, attempt identifiers, and an appeal or ambiguity rule. Local reputation may be safer than global automatic slashing for many events.

12.6 Economic optimality

The experiment reports success, volume, fees, imbalance, failed attempts, and added capital, but it does not calculate return on capital or on-chain channel cost. A 120,000-satoshi connector may be uneconomic even when it raises success. The correct decision depends on fee revenue, user value, capital cost, expected duration, close cost, and opportunity cost.

12.7 Reconstruction provenance

This is a reconstruction after a filesystem incident. The original manuscript, equations, code, bibliography, PDF, and Git objects were recovered as null-filled or invalid files. The present draft uses surviving filenames, package metadata, 19 user prompts, the public title of the earlier ghost-node work, and authoritative Lightning literature. It should not be represented as verbatim restoration of the lost draft.

13. Conclusion

Lightning liquidity is a graph-state problem before it is a marketplace slogan. Public topology says where channels exist; private directional state says what can move now. Payments rewrite that state. Rebalances, openings, leases, and cross-system claims are different transitions and should be typed accordingly.

The connector calculus makes a missing intervention explicit as a ghost plan and refuses to count it until it compiles into capital, policy, horizon, and bond. The resulting framework links graph theory to operational accountability. It also gives autonomous agents a constructive role: they may transform observed demand into funded service, but only under reward caps and evidence that prevent identity multiplication and wash flow from masquerading as liquidity.

The experiments support three narrow conclusions. Payment rewrites conserve modeled channel capacity. Added cross-cut capital improves delivery and resilience in the specified synthetic graphs. Capital-keyed rewards are exactly invariant to identity splitting. The experiments do not show that the simple demand-cut heuristic is better than random placement, nor do they establish mainnet economics.

The next research gate is a time-sliced, holdout evaluation on realistic Lightning snapshots with calibrated hidden balances, MPP, HTLC timing, on-chain cost, and adversarial demand. If the connector rule continues to improve delivery after capital and privacy costs, the bonding layer can move from illustrative records to regtest service contracts. ZK proofs, Ark adapters, and asset settlement belong after that gate, not before it.

Appendix A. Formal Proofs and Derived Mechanics

A.1 State polytope

For fixed topology E and capacities C , the directional state space is the product:

Equation

$$\Omega(C) = \text{product}_{(e \in E)} [0, c_e].$$

Choose one orientation $u_e \rightarrow v_e$ for each channel and store $x_e = b_{(u_e, v_e)}$. The reverse balance is derived as $c_e - x_e$. A feasible path rewrite adds a signed vector δ to x . For an edge traversed with its stored orientation, $\delta_e = -q$; against it, $\delta_e = +q$; otherwise $\delta_e = 0$. Feasibility is exactly the condition $x + \delta$ in Ω after reserves and locks.

This representation removes a redundant variable per channel and makes conservation structural. The implementation retains a Channel object because endpoint direction, fees, locks, and readability matter.

A.2 Proof of path conservation

Let p contain edges e_1 through e_k . Each edge rewrite changes only its two directional balances and preserves their sum. Nontraversed edges are unchanged. Summing channel capacities over E before and after the path yields the same value. Since the complete path is validated before mutation, a rejected route leaves the entire state unchanged.

A.3 Commutativity of edge-disjoint rewrites

Let p and r be two feasible paths whose channel sets are disjoint. Their rewrite vectors have disjoint support. Therefore:

Equation

$$T_p(T_r(x)) = x + \delta_r + \delta_p = T_r(T_p(x)).$$

This does not imply that concurrent Lightning payments are generally commutative. Paths sharing a channel compete for directional liquidity, and feasibility after the first rewrite may differ. Production implementations need reservation and atomicity rules.

A.4 Circular connector conservation

For a closed vertex sequence $v_0, \dots, v_k$ with $v_k = v_0$, the node incidence of a uniform modeled flow q is zero: each occurrence contributes one incoming and one outgoing q . Channel-side balances change, but the connector introduces no source or sink. With real fees, the initiator supplies the fee difference; the cycle is not value-free.

A.5 Ghost compilation safety

Define $\text{Compile}(g, S)$ as a partial function. If any predicate fails - unsupported mechanism, duplicate capital, absent consent, insufficient fee budget, unsafe expiry, missing bond, or concentration breach - Compile is undefined and S is unchanged. If it succeeds, it returns a connector κ with concrete economic fields. $\text{Apply}(\kappa, S)$ is a separate transition. This two-step structure is equivalent to a plan/commit boundary.

A.6 Identity-split theorem

Let claims for capital c be identities i in M_c . The allocator first computes R_c without using $|M_c|$. It then chooses nonnegative shares r_i whose sum is R_c . For any split or merge that leaves c and its service weight unchanged:

Equation

$$\sum_{(i \in M_c)} r_i = R_c.$$

The theorem assumes that duplicate-capital detection is correct. If an operator can represent the same economic exposure under different capital identifiers, the premise fails. If an operator owns additional real capital, additional reward may be legitimate but concentration controls remain necessary.

Appendix B. Experiment Protocols

B.1 P1 conservation campaign

1. Generate seed-7 two-cluster state.
2. Draw 600 random source-target pairs and bounded integer amounts.
3. Enumerate up to four shortest simple paths.
4. Apply the first feasible path or record rejection.
5. After each success, assert channel sum, bounds, lock bounds, and total capacity.
6. Report applied and rejected counts.

Falsifier: any assertion failure or capacity change.

B.2 P2 connector campaign

1. For seeds 0-23, generate identical baseline, random, and ghost states.
2. Generate 360 demands per seed.
3. Add no channel, equal-capital random cross-cut channel, or demand-cut channel.
4. Route adaptively with up to eight candidates.
5. Report paired seed-level success deltas and normal 95% intervals.

Falsifier for baseline claim: mean paired ghost-baseline delta ≤ 0 . **Falsifier for placement claim:** mean paired ghost-random interval includes or falls below 0. The observed placement claim is therefore not supported.

B.3 Privacy frontier

1. Reuse baseline states and demands.
2. Run public, adaptive, and oracle knowledge modes.
3. Count failed candidate paths before success or exhaustion.
4. Plot mean success against mean failed-attempt proxy.

Boundary: no privacy bits, identities, or network packets are measured.

B.4 Synthetic lock campaign

1. Construct baseline or ghost state.
2. Compute undirected edge betweenness.
3. Select two highest-scoring edges.
4. Mark 90% of each directional balance unavailable.
5. Run the same 360 demands.

Boundary: this is resource removal, not an HTLC attack implementation.

B.5 Sybil campaign

1. Define one capital identifier with 100,000 capital units and score 0.9.
2. Split the claim across m identities for m in $\{1,2,4,8,16,32\}$.
3. Allocate a fixed 100,000-unit capital-keyed pool.
4. Compare with a 100,000-unit per-identity control.

Falsifier: capital-keyed coalition total differs across m.

Appendix C. Red-Team Matrix

Vector	Attacker advantage	Observable symptom	Primary control	Residual risk
Identity splitting	Multiplies grants or votes	Many claims share capital or behavior	Capital-keyed pool	Distinct but correlated capital
Liquidity mirage	Attracts flow without service	Quote-success gap, rapid withdrawal	Unique exposure proof, horizon	Directional availability remains private
Jamming	Locks scarce route resources	Long-lived failures, peer concentration	Local resource pricing and diversity	Attribution ambiguity
Probing	Learns private balances	Repeated boundary searches	Retry caps, coarse errors, splitting	Efficiency loss
Wash flow	Manufactures reward volume	Reciprocal/circular low-value flow	Net external demand metric	Colluding customer identities
Demand poisoning	Induces bad channel placement	Early burst then regime shift	Holdout evaluation, signal bond	Genuine nonstationarity
Cut capture	Gains surveillance or censorship position	Correlated central connectors	Failure-domain caps	Hidden common control
False slashing	Steals or burns operator bond	Disputed witness events	Objective receipts, ambiguity rule	Complex multi-hop causality
Model monoculture	Correlated agent action	Synchronized rebalances and quotes	Policy diversity and rate limits	Shared upstream data
External adapter failure	Breaks virtual connector settlement	VTXO/swap timeout or exit stress	Explicit adapter and exit proof	Cross-system congestion

Appendix D. Proof Receipts and Data Dictionary

The machine-readable output uses three layers.

Experiment rows contain seed, strategy, attempts, successes, volumes, fees, failed-attempt proxy, imbalance, and connector capital. **Summary records** contain means, paired deltas, and limitations. **Proof cards** bind a human-readable claim to the relevant evidence and disposition.

Each proof-suite run writes `witness_receipt.json` with:

```

experiment_id
configuration_file
configuration_hash
result_file
result_hash
live_network = false
zk_proof = false

```

The receipt binds the exact proof configuration to the exact `summary.json` bytes with SHA-256. It intentionally excludes wallet keys, RPC credentials, prompts, model identifiers, and strategy payloads. Hashes support integrity and reproduction; they do not prove that the simulator is an adequate model.

Appendix E. Reproduction Checklist

1. Use Python 3.11 or later.
2. Install the project dependencies and pytest.
3. Set PYTHONPATH to `src`.
4. Run `python -m pytest -q`.

5. Run `python -m spiral_ln.experiment --output output/experiments --seeds 24`.
6. Compare `summary.json` with the tables in Section 9.
7. Build the PDF with `python -m spiral_ln.paper`.
8. Render every page and inspect equations, tables, figures, references, headers, and page numbers.

References

- Ark protocol contributors. 2026. **Ark Protocol Technical Documentation: Virtual Transaction Outputs and Connectors**. <https://ark-protocol.org/intro/>
- Douceur, John R. 2002. "The Sybil Attack." In **Peer-to-Peer Systems**, 251-260.
- Ford, Lester R., and Delbert R. Fulkerson. 1956. "Maximal Flow Through a Network." **Canadian Journal of Mathematics** 8: 399-404.
- Lightning Network Specifications contributors. 2026. **Basis of Lightning Technology Specifications**. <https://github.com/lightning/bolts>
- Mazumdar, Subhra, Prabal Banerjee, Abhinandan Sinha, Sushmita Ruj, and Bimal Roy. 2022. "Strategic Analysis of Griefing Attack in Lightning Network." arXiv:2203.10533.
- Nisslmueller, Utz, Klaus-Tycho Foerster, Stefan Schmid, and Christian Decker. 2020. "Toward Active and Passive Confidentiality Attacks on Cryptocurrency Off-Chain Networks." arXiv:2003.00003.
- Pickhardt, Rene, and Stefan Richter. 2021. "Optimally Reliable and Cheap Payment Flows on the Lightning Network." arXiv:2107.05322.
- Pickhardt, Rene, Sergei Tikhomirov, Alex Biryukov, and Mariusz Nowostawski. 2021. "Security and Privacy of Lightning Network Payments with Uncertain Channel Balances." arXiv:2103.08576.
- Poon, Joseph, and Thaddeus Dryja. 2016. **The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments**. <https://lightning.network/lightning-network-paper.pdf>
- Romiti, Matteo, Friedhelm Victor, Pedro Moreno-Sanchez, Peter Sebastian Nordholt, Bernhard Haslhofer, and Matteo Maffei. 2020. "Cross-Layer Deanonimization Methods in the Lightning Protocol." arXiv:2007.00764.
- Rubio, Lihki, Dugan, and Pizarro. 2020. **Graph Re-write Using Ghost Nodes for Target Value Rebalancing**. U.S. Patent Application 16/920,416.
- Sharma, Piyush Kumar, Devashish Gosain, and Claudia Diaz. 2022. "On the Anonymity of Peer-to-Peer Network Anonymity Schemes Used by Cryptocurrencies." arXiv:2201.11860.
- Shikhelman, Clara, and Sergei Tikhomirov. 2022. "Unjamming Lightning: A Systematic Approach." IACR ePrint 2022/1454.
- Tikhomirov, Sergei, Pedro Moreno-Sanchez, and Matteo Maffei. 2020. "A Quantitative Analysis of Security, Anonymity and Scalability for the Lightning Network." **IEEE EuroS&P Workshops**, 387-396.
- Tikhomirov, Sergei, Rene Pickhardt, Alex Biryukov, and Mariusz Nowostawski. 2020. "Probing Channel Balances in the Lightning Network." arXiv:2004.00333.
- van Dam, Gijs, Rabiah Abdul Kadir, Sharifah Md Yasin, and Halimah Badioze Zaman. 2026. "Payment Splitting in Lightning Network as a Mitigation Against Balance Discovery Attacks." **Blockchain: Research and Applications**. doi:10.1016/j.bcr.2026.100500.
- Zabka, Philipp, Klaus-Tycho Foerster, Stefan Schmid, and Christian Decker. 2022. "A Centrality Analysis of the Lightning Network." arXiv:2201.07746.